

# ShadowTalk Developer Onboarding Guide

Welcome to the ShadowTalk engineering team! This guide is designed to help you, as a new pair programmer, get up to speed quickly with the project's architecture, tech stack, and core workflows.

---

## 1. Project Overview

**ShadowTalk** is an Agentic AI workspace featuring Mission Control, 30+ tools, and optional on-device privacy. It allows users to manage workflows intelligently and even create a “Shadow Twin”—an autonomous AI clone that can interact with the public on the user’s behalf.

## 2. Tech Stack

### Frontend & Core

- **React (v18)**: Core UI library.
- **TypeScript**: Strictly typed JavaScript for better developer experience and safety.
- **Vite**: Ultra-fast build tool and development server.
- **React Router DOM**: Client-side routing.
- **Zustand & React Context**: State management. Zustand is generally used for global, reactive stores, while Context is used for providers (like Auth).

### Styling & UI

- **Tailwind CSS**: Utility-first CSS framework for rapid styling.
- **shadcn/ui (Radix UI)**: Unstyled, accessible UI primitives. Our components (buttons, dialogs, dropdowns) are built on top of these.
- **Framer Motion**: Used extensively for micro-interactions, page transitions, and fluid animations.
- **Lucide React**: Our standard iconography library.

### Backend & Cloud

- **Firebase**: We heavily utilize Firebase for:
  - **Hosting**: Hosting the production application.

- **Auth:** Managing user sessions and authentication.
- **Firestore:** NoSQL database for storing user data, chat histories, etc.
- **Supabase:** Also present in the stack (often used for vector embeddings or specialized relational data needs).

## AI & Integrations

- **Web-LLM:** Enables optional on-device, local AI execution for maximum privacy.
- **Anthropic SDK:** Used for cloud-based LLM inference.
- **Composio:** Integration layer for external tools and agents.

## 3. Directory Structure

The codebase follows a standard feature-based React architecture. All primary code lives within the `src/` directory.

```
src/
├── assets/           # Static assets (images, fonts, global styles)
├── components/       # Reusable UI components
│   ├── ui/          # Base shadcn/ui components (Button, Input, etc.)
│   ├── chat/        # Chat-specific components (MessageBubble, ChatInput)
│   └── settings/     # Components used in the settings dashboards
├── contexts/        # React Context providers (e.g., AuthProvider)
├── hooks/           # Custom React hooks (e.g., useConversationManager)
├── integrations/    # Third-party service integrations (Composio, etc.)
├── lib/             # Utility functions, helpers, and API wrappers (e.g.,
├── pages/           # Top-level route components (Pages)
├── types/           # Global TypeScript type definitions
├── App.tsx          # Root component and Router configuration
└── main.tsx         # React DOM entry point
```

## 4. Key Features & How They Work

### The Chat Interface (Mission Control)

- **Location:** `src/pages/ChatbotPage.tsx`
- **Logic:** Handled largely by `src/hooks/useConversationManager.ts`. The `Message` interface dictates the shape of chat data.
- **UI:** The chat UI is built using `MessageBubble.tsx`, which parses Markdown and renders AI and user messages with specific styling.

### Shadow Twin (Viral Growth Feature)

Users can generate a public-facing AI clone of themselves.

- **Setup:** Users configure their twin in `src/pages/ShadowTwinSettingsPage.tsx` (Route: `/shadow-twin`).
- **Public Portal:** Guest users can chat with a user's twin at `src/pages/PublicShadowTwinChat.tsx` (Route: `/t/:username`).
- **Under the Hood:** The app injects a specific system prompt into the AI stream: *"You are acting as the Shadow Twin for [Username]. You must speak and act entirely on their behalf..."* and applies special UI theming (purple colorways, Sparkles icon) to twin messages.

### Routing ( `App.tsx` )

We use `react-router-dom`. Routes are wrapped in an `AuthProvider` for protected pages, while public pages (like the Landing Page and Public Shadow Twin Chat) bypass authentication requirements.

---

## 5. Development Workflow

### Getting Started

#### 1. Install Dependencies:

```
npm install
```

2. **Environment Variables:** Copy the example environment file and fill in your Firebase/API keys.

```
cp env.example .env
```

3. **Run the Dev Server:**

```
npm run dev
```

The app will be available at `http://localhost:5173`.

## Code Style Guidelines

- **Functional Components:** Always use React functional components with Hooks.
- **Tailwind:** Prefer Tailwind utility classes over custom CSS in `index.css` whenever possible.
- **Aesthetics First:** We pride ourselves on a premium UI. When building new components, ensure they utilize our design tokens (glassmorphism, subtle borders, `framer-motion` enter/exit animations).

## Deployment

Deployment is handled via the Firebase CLI. When a feature is ready for production:

```
# Build the production bundle
npm run build

# Deploy to Firebase Hosting
npx firebase deploy --only hosting
```

---

## 6. Your First Day Checklist

- ☐ Get the app running locally ( `npm run dev` ).

- ☐ Create a test account via the Auth flow.
- ☐ Navigate to the Chatbot page and send a message.
- ☐ Create your own Shadow Twin in the settings and test the public URL ( `/t/your-username` ).
- ☐ Review `src/hooks/useConversationManager.ts` to understand how message state is tracked.

Welcome aboard! If you have any questions, just ask your pair-programming agent!